JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class RecursiveAction

java.lang.Object
    java.util.concurrent.ForkJoinTask<Void>
        java.util.concurrent.RecursiveAction

All Implemented Interfaces:

Serializable, Future<Void>

```
public abstract class RecursiveAction
extends ForkJoinTask<Void>
```

A recursive resultless ForkJoinTask. This class establishes conventions to parameterize resultless actions as `Void` `ForkJoinTasks`. Because `null` is the only valid value of type `Void`, methods such as `join` always return `null` upon completion.

Sample Usages. Here is a simple but complete ForkJoin sort that sorts a given `long[]` array:

```
 static class SortTask extends RecursiveAction {
   final long[] array; final int lo, hi;
   SortTask(long[] array, int lo, int hi) {
     this.array = array; this.lo = lo; this.hi = hi;
   }
   SortTask(long[] array) { this(array, 0, array.length); }
   protected void compute() {
     if (hi - lo < THRESHOLD)
       sortSequentially(lo, hi);
     else {
       int mid = (lo + hi) >>> 1;
       invokeAll(new SortTask(array, lo, mid),
                 new SortTask(array, mid, hi));
```

```
      merge(lo, mid, hi);
    }
  }
  // implementation details follow:
  static final int THRESHOLD = 1000;
  void sortSequentially(int lo, int hi) {
    Arrays.sort(array, lo, hi);
  }
  void merge(int lo, int mid, int hi) {
    long[] buf = Arrays.copyOfRange(array, lo, mid);
    for (int i = 0, j = lo, k = mid; i < buf.length; j++)
      array[j] = (k == hi || buf[i] < array[k]) ?
        buf[i++] : array[k++];
  }
}
```

You could then sort `anArray` by creating `new SortTask(anArray)` and invoking it in a ForkJoinPool. As a more concrete simple example, the following task increments each element of an array:

```
class IncrementTask extends RecursiveAction {
  final long[] array; final int lo, hi;
  IncrementTask(long[] array, int lo, int hi) {
    this.array = array; this.lo = lo; this.hi = hi;
  }
  protected void compute() {
    if (hi - lo < THRESHOLD) {
      for (int i = lo; i < hi; ++i)
        array[i]++;
    }
    else {
      int mid = (lo + hi) >>> 1;
      invokeAll(new IncrementTask(array, lo, mid),
                new IncrementTask(array, mid, hi));
    }
  }
}
```

The following example illustrates some refinements and idioms that may lead to better performance: RecursiveActions need not be fully recursive, so long as they maintain the basic divide-and-conquer approach. Here is a class that sums the squares of each element of a double array, by subdividing out only the right-hand-sides of repeated divisions by two, and keeping track of them with a chain of `next` references. It uses a dynamic threshold based on method `getSurplusQueuedTaskCount`, but counterbalances potential excess partitioning by directly performing leaf actions on unstolen tasks rather than further subdividing.

```
double sumOfSquares(ForkJoinPool pool, double[] array) {
```

```
  int n = array.length;
  Applyer a = new Applyer(array, 0, n, null);
  pool.invoke(a);
  return a.result;
}

class Applyer extends RecursiveAction {
  final double[] array;
  final int lo, hi;
  double result;
  Applyer next; // keeps track of right-hand-side tasks
  Applyer(double[] array, int lo, int hi, Applyer next) {
    this.array = array; this.lo = lo; this.hi = hi;
    this.next = next;
  }

  double atLeaf(int l, int h) {
    double sum = 0;
    for (int i = l; i < h; ++i) // perform leftmost base step
      sum += array[i] * array[i];
    return sum;
  }

  protected void compute() {
    int l = lo;
    int h = hi;
    Applyer right = null;
    while (h - l > 1 && getSurplusQueuedTaskCount() <= 3) {
      int mid = (l + h) >>> 1;
      right = new Applyer(array, mid, h, right);
      right.fork();
      h = mid;
    }
    double sum = atLeaf(l, h);
    while (right != null) {
      if (right.tryUnfork()) // directly calculate if not stolen
        sum += right.atLeaf(right.lo, right.hi);
      else {
        right.join();
        sum += right.result;
      }
      right = right.next;
    }
    result = sum;
  }
}
```

Since:

1.7

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`RecursiveAction()`	

Method Summary

Concrete Methods **Abstract Methods**

Modifier and Type	Method	Description
protected abstract void	`compute()`	The main computation performed by this task.
protected boolean	`exec()`	Implements execution conventions for RecursiveActions.
`Void`	`getRawResult()`	Always returns `null`.
protected void	`setRawResult(Void mustBeNull)`	Requires null completion value.

Methods inherited from class java.util.concurrent.ForkJoinTask

adapt, adapt, adapt, cancel, compareAndSetForkJoinTaskTag, complete, completeExceptionally, fork, get, get, getException, getForkJoinTaskTag, getPool, getQueuedTaskCount, getSurplusQueuedTaskCount, helpQuiesce, inForkJoinPool, invoke, invokeAll, invokeAll, invokeAll, isCancelled, isCompletedAbnormally, isCompletedNormally, isDone, join, peekNextLocalTask, pollNextLocalTask, pollTask, quietlyComplete, quietlyInvoke, quietlyJoin, reinitialize, setForkJoinTaskTag, tryUnfork

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

RecursiveAction

```
public RecursiveAction()
```

Method Detail

compute

```
protected abstract void compute()
```

The main computation performed by this task.

getRawResult

```
public final Void getRawResult()
```

Always returns `null`.

Specified by:

`getRawResult` in class `ForkJoinTask<Void>`

Returns:

`null` always

setRawResult

```
protected final void setRawResult(Void mustBeNull)
```

Requires null completion value.

Specified by:

`setRawResult` in class `ForkJoinTask<Void>`

Parameters:

`mustBeNull` - the value

exec

```
protected final boolean exec()
```

Implements execution conventions for RecursiveActions.

Specified by:

`exec` in class `ForkJoinTask<Void>`

Returns:

`true if this task is known to have completed normally`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.